



İSTANBUL UNIVERSITY  
C E R R A H P A Ş A

T. C.

İSTANBUL ÜNİVERSİTESİ-CERRAHPAŞA  
MÜHENDİSLİK FAKÜLTESİ  
Bilgisayar Mühendisliği Bölümü

• • •

BİLGİSAYAR MİMARİSİ DERSİ – DÖNEM PROJESİ



Genişletilmiş Komut Kümeli  
**Multi-Cycle MIPS İşlemci Tasarımı**  
Tasarım, Genişletme ve Doğrulama

Grup Üyeleri

| Öğrenci Adı            | No         | Sorumluluk Alanı                   |
|------------------------|------------|------------------------------------|
| Arda AYDIN             | 1306230121 | Datapath Tasarımı                  |
| Mehmet BAL             | 1306220020 | Entegrasyon & Dokümantasyon        |
| Melih Can İÇÖZ         | 1306220042 | Komut seti ve assembler geliştirme |
| Abdullah Taha ÜSTÜNSOY | 1306230117 | Test ve Doğrulama                  |
| Ahmet Melih ÜSTÜNER    | 1306230119 | Kontrol Birimi Tasarımı            |

Geliştirme Ortamı: ModelSim Intel FPGA Edition 2020.1 · VHDL-2008 · Python 3

Avcılar Yerleşkesi, İstanbul

**Haziran 2026**

## 1. Projenin Amacı ve Kapsamı

Bu çalışmanın amacı, MIPS komut kümesi mimarisini temel alan ve bir sonlu durum makinesi (FSM) tarafından denetlenen çok çevrimli (multi-cycle) bir işlemcinin VHDL ile tasarlanması, ek komutlarla genişletilmesi ve kapsamlı testbench'lerle doğrulanmasıdır. Aşağıdaki başlıklar projenin temel bileşenlerini özetler:

### Çok çevrimli işlemci tasarımı

Her komut, tamamlanması için gereken çevrim sayısına göre IF, ID, EX, MEM ve WB aşamalarına bölünmüştür. Bu yaklaşım kritik yol uzunluğunu kısaltarak daha yüksek saat frekansına olanak tanır.

### FSM tabanlı denetim

Denetim birimi VHDL'de iki süreçli (two-process) FSM olarak kurgulanmıştır. Senkron durum geçişi ile bileşimsel çıkış üretimi ayrı süreçlerde ele alınmış, latch oluşumunu önlemek için tüm çıkışlara başlangıçta güvenli varsayılan değerler atanmıştır. FSM toplam 23 durumdan oluşur.

### Genişletilmiş komut kümesi

Standart MIPS komutlarına ek olarak yedi yeni komut tasarlanmıştır: `mul`, `swap`, `loadi`, `bgt`, `push`, `pop` ve `addi3`. Bu komutlar hem donanım hem de assembler düzeyinde tam olarak desteklenir.

### Assembler ve doğrulama

Python 3 ile yazılmış iki geçişli (two-pass) bir assembler, assembly kaynağını ModelSim ile uyumlu `$readmemh` biçimindeki `.hex` dosyasına dönüştürür. Doğrulama tarafında on üç adet kendi kendini denetleyen (self-checking) testbench ile birim ve bütünleşme testleri yürütülmüş, hiçbir testbench'te hata gözlenmemiştir.

## 2. Sistem Mimarisi

İşlemci `cpu.vhd` içinde yapısal (structural) bir mimariyle üç ana bileşeni birleştirir: denetim birimi (control\_unit), ALU denetimi (alu\_control) ve veri yolu (datapath). Modüller arasındaki sinyal akışı aşağıda özetlenmiştir.

```

control_unit (FSM)
├── denetim sinyalleri ────> datapath
├── alu_op (2-bit) ────> alu_control
alu_control
├── alu_ctrl (4-bit) ────> datapath
datapath
├── opcode, funct, zero, neg → control_unit / alu_control
└── PC + Memory + IR + Register File + ALU + ara registerlar

```

### 2.1 Program Counter (PC) — src/pc.vhd

32-bit senkron register. Çok çevrimli tasarımda PC her saat kenarında güncellenmez; yalnızca `pc_write='1'` olduğunda `pc_next` değerini yükler. `reset='1'` iken `PC=0` olur.

| Port     | Yön   | Genişlik | Açıklama                          |
|----------|-------|----------|-----------------------------------|
| clk      | Giriş | 1        | Sistem saati                      |
| reset    | Giriş | 1        | Senkron reset; '1' iken PC=0      |
| pc_write | Giriş | 1        | Enable; '1' iken pc_next yüklenir |
| pc_next  | Giriş | 32       | Yüklenecek adres                  |
| pc_out   | Çıkış | 32       | Mevcut PC değeri                  |

## 2.2 Instruction Register (IR) — src/instruction\_register.vhd

IF aşamasında bellekten okunan komutu kilitler ve 32-bit sözcüğü alanlarına ayırır. Von Neumann mimarisinde bellek LW/SW sırasında da kullanıldığından komutun kaybolmaması için IR zorunludur. `ir_write='1'` yalnızca IF aşamasında etkindir.

| Alan      | Bit Aralığı | Açıklama                           |
|-----------|-------------|------------------------------------|
| opcode    | [31:26]     | Komut tipi                         |
| rs        | [25:21]     | Kaynak register 1                  |
| rt        | [20:16]     | Kaynak register 2 / hedef (I-type) |
| rd        | [15:11]     | Hedef register (R-type)            |
| funct     | [5:0]       | Fonksiyon kodu (R-type)            |
| immediate | [15:0]      | I-type immediate                   |
| address   | [25:0]      | J-type hedef adresi                |

## 2.3 Register File — src/register\_file.vhd

32 adet 32-bit register içerir. İki asenkron okuma portu (anlık) ve bir senkron yazma portu (saat kenarı) bulunur. `$0` (\$zero) her zaman 0'dır: `write_reg /= "00000"` koşulu donanım içinde uygulanır. `$sp` register 29'dur ("11101"). Testbench gözlemi için eklenen debug portu (`dbg_reg/dbg_data`) donanım mantığını etkilemez.

## 2.4 Memory — src/memory.vhd

$256 \times 32$ -bit sözcük = 1024 bayt. Von Neumann düzeninde tek bellek birimi komut ve veriyi birlikte tutar. Bayt adresi girilir; sözcük indeksi `address(9 downto 2)` ile elde edilir. Asenkron okuma, senkron yazma yapılır. `INIT_FILE` generic parametresiyle hex dosyadan yüklenebilir.

## 2.5 ALU — src/alu.vhd

Saat içermeyen bileşimsel bir devredir ve yedi işlemi destekler:

| alu_ctrl | İşlem                   | Kullanan komutlar                          |
|----------|-------------------------|--------------------------------------------|
| 0000     | AND                     | and, andi                                  |
| 0001     | OR                      | or, ori                                    |
| 0010     | ADD (signed)            | add, addi, lw, sw, addi3, push, pop, loadi |
| 0011     | XOR                     | xor, xori                                  |
| 0110     | SUB (signed)            | sub, beq, bne, bgt, slt, push              |
| 0111     | SLT                     | slt, slti                                  |
| 1000     | MUL (32×32, alt 32-bit) | mul                                        |

## 2.6 ALU Control — src/alu\_control.vhd

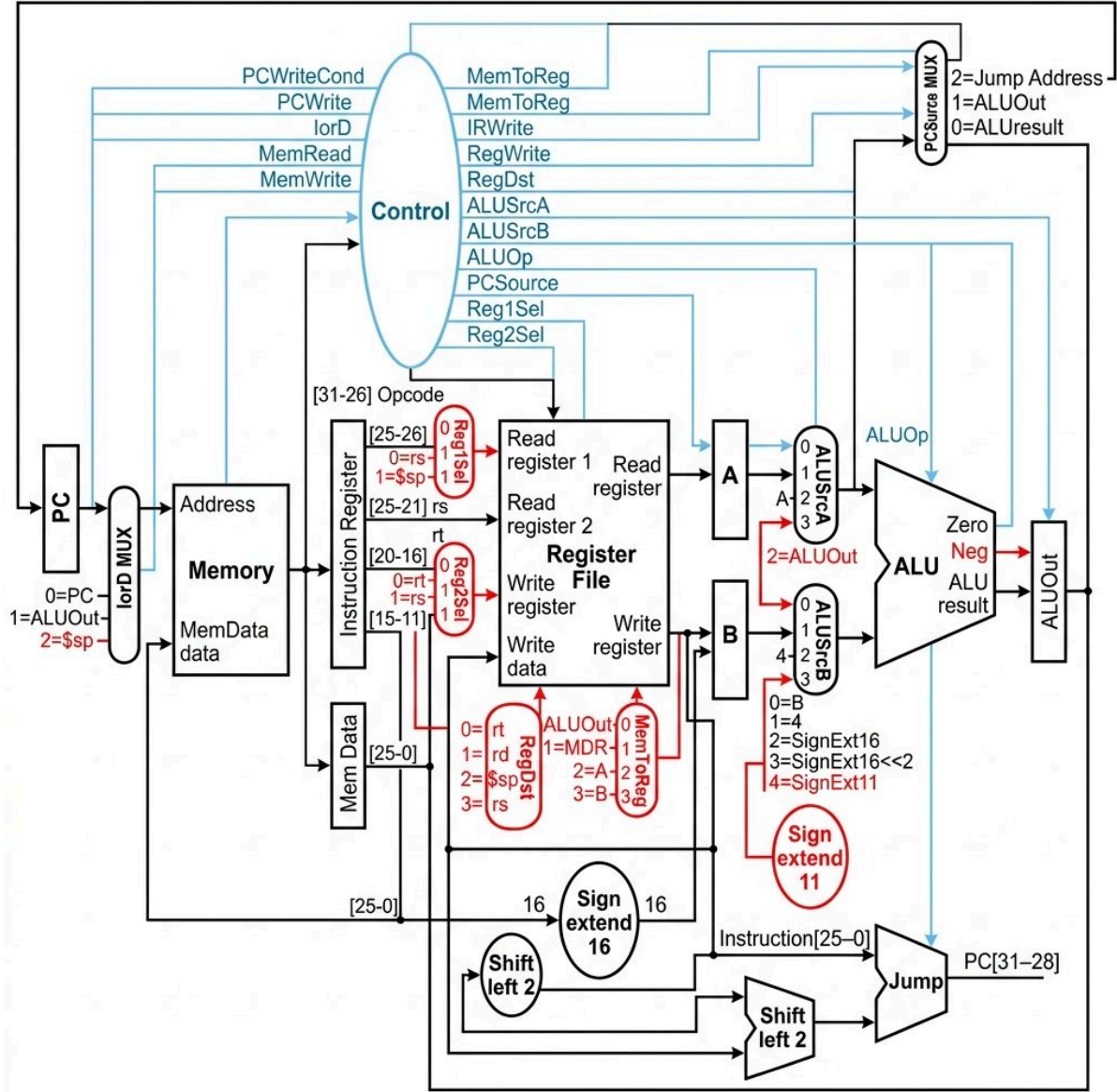
2-bit `alu_op` + opcode + funct girdilerini 4-bit `alu_ctrl` çıkışına çözer. Bu iki katmanlı çözümleme sayesinde FSM her R-type funct kodunu bilmek zorunda kalmaz: `alu_op="00"` → ADD, `"01"` → SUB, `"10"` → komuta göre çözülür.

## 2.7 Datapath — src/datapath.vhd

Tüm donanım bileşenlerini MUX'larla birleştiren veri yoludur. Kendisi hesap yapmaz; verinin nereye akacağı tümüyle dış denetim sinyalleriyle belirlenir. Ara registerlar (A, B, ALUOut, MDR) her çevrimde yüklenir (en= '1' sabit).

### 3. Veri Yolu Tasarımı

Veri yolu, derste kullanılan standart multi-cycle MIPS datapath'i temel alınarak oluşturulmuş; yedi yeni komutu desteklemek için gereken ek donanım birimleri ve yolları bunun üzerine eklenerek genişletilmiştir. Aşağıdaki diyagramda standart bileşenler siyah, bu proje kapsamında eklenen birimler ve sinyal yolları kırmızı ile gösterilmiştir.



Şekil 3.1 — Genişletilmiş multi-cycle MIPS veri yolu. Siyah bileşenler standart datapath'i, kırmızı ile vurgulanan birimler ve yollar (Reg1Sel/Reg2Sel MUX, IorD girişi 2, RegDst girişleri 2-3, MemToReg girişleri 2-3, ALUSrcA girişi 2, ALUSrcB girişi 4 ve Sign extend 11, ALU'dan Control'e Neg çıkışı) yeni komutlar için yapılan donanım genişletmelerini gösterir.

#### 3.1 Eklenen donanım birimleri ve yolları

Diyagramda kırmızı ile vurgulanan birimlerin işlevleri aşağıda özetlenmiştir:

- **Reg1Sel ve Reg2Sel MUX:** **push** ve **pop** işlemlerinde stack pointer (\$sp) ile kaynak register'ın seçimi için eklendi.
- **IorD MUX (giriş 2):** **pop** işleminde bellekten veri çekilirken adresin \$sp register'ından (A) okunması için eklendi.
- **RegDst MUX (giriş 2 ve 3):** yığın işlemleri için \$sp, **swap** işlemi için rs register hedefleri eklendi.
- **MemToReg MUX (giriş 2 ve 3):** **swap** komutunda register değerlerinin takası için A ve B geçici register çıkışları eklendi.
- **ALUSrcA MUX (giriş 2):** **addi3** komutundaki ardışık toplama için ALUOut geri besleme yolu eklendi.
- **ALUSrcB MUX (giriş 4) ve Sign extend 11:** **addi3** komutundaki 11-bit işaretli anlık değeri (imm11) genişleterek ALU'ya aktarmak için eklendi.
- **Neg çıkışı (ALU → Control):** **bgt** komutunda büyüklük karşılaştırması ( $A - B > 0$ , yani işaret biti denetimi) için eklendi.

### 3.2 MUX yapıları

| MUX      | Denetim sinyali | Seçenekler                                           |
|----------|-----------------|------------------------------------------------------|
| IorD     | ior_d[1:0]      | 00=PC · 01=ALUOut (lw/sw) · 10=A (\$sp, pop)         |
| ALUSrcA  | alu_src_a[1:0]  | 00=PC · 01=A · 10=ALUOut (addi3 geri besleme)        |
| ALUSrcB  | alu_src_b[2:0]  | 000=B · 001=4 · 010=sx16 · 011=sx16<<2 · 100=sx11    |
| PCSource | pc_source[1:0]  | 00=ALU (PC+4) · 01=ALUOut (branch) · 10=jump         |
| RegDst   | reg_dst[1:0]    | 00=rt · 01=rd · 10=\$sp · 11=rs (swap)               |
| MemToReg | mem_to_reg[1:0] | 00=ALUOut · 01=MDR · 10=A (swap→rt) · 11=B (swap→rs) |

### 3.3 İşaret genişletme ve dallanma/atlama adresi

signext16 = resize(signed(ir\_imm), 32). Dallanma hedefi:  $(PC+4) + \text{signext16} \ll 2$ . addi3 için signext11 = resize(signed(ir\_imm(10 downto 0)), 32). Atlama adresi:  $\text{jump\_addr} = PC[31:28] \& \text{ir\_addr} \& "00"$ .

**Tasarım kararı (anlık değer genişletme):** Bu tasarımda tüm I-type komutlar (mantıksal andi/ori/xori dâhil) tek bir ortak 16-bit işaret-genişletme (signext16) yolunu kullanır. Standart MIPS'te andi/ori/xori anlık değeri sıfır-genişletilir (zero-extend); burada donanımı sadeleştirmek için ayrı bir zero-extend yolu eklenmemiş, işaret-genişletme tercih edilmiştir. Bu fark yalnızca en yüksek biti (bit 15) 1 olan anlık değerlerde sonucu etkiler; testlerde kullanılan küçük pozitif anlık değerlerde davranış standart MIPS ile birebir aynıdır.

### 3.4 Yığın (stack) mekanizması

Aşağı doğru büyüyen bir yığın kullanılır; \$sp register 29'dur. PUSH:  $\$sp -= 4$ ;  $\text{MEM}[\$sp] = rs$ . POP:  $rd = \text{MEM}[\$sp]$ ;  $\$sp += 4$ .

| Komut | Durum       | İşlem                                                                 | Anahtar sinyaller       |
|-------|-------------|-----------------------------------------------------------------------|-------------------------|
| PUSH  | S_PUSH_ADDR | ALU: $\$sp - 4 \rightarrow \text{ALUOut}$                             | reg1_sel=1 (A=\$sp)     |
| PUSH  | S_PUSH_MEM  | $\text{MEM}[\text{ALUOut}] \leftarrow B(rs)$                          | ior_d=01, mem_write=1   |
| PUSH  | S_PUSH_SP   | $\$sp \leftarrow \text{ALUOut}$                                       | reg_dst=10, reg_write=1 |
| POP   | S_POP_READ  | $\text{MDR} \leftarrow \text{MEM}[\$sp]$ ; $\text{ALUOut} = \$sp + 4$ | ior_d=10, mem_read=1    |

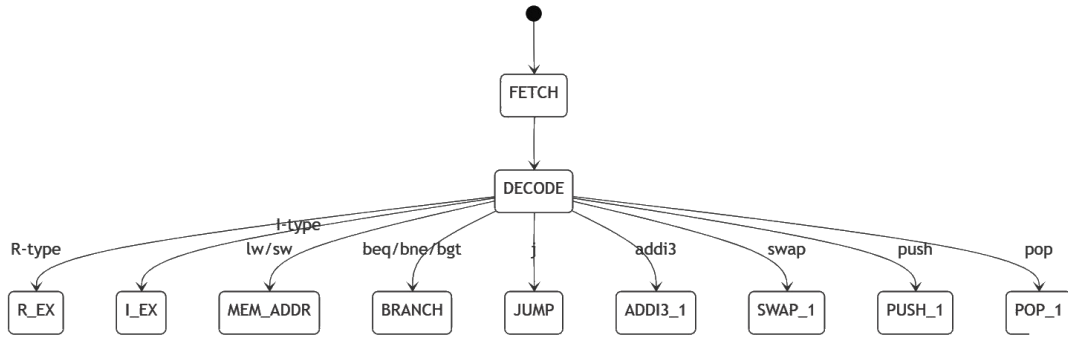
| Komut | Durum    | İşlem                    | Anahtar sinyaller               |
|-------|----------|--------------------------|---------------------------------|
| POP   | S_POP_WB | $rt \leftarrow MDR$      | $mem\_to\_reg=01, reg\_write=1$ |
| POP   | S_POP_SP | $\$sp \leftarrow ALUOut$ | $reg\_dst=10, reg\_write=1$     |

## 4. FSM Tabanlı Denetim Birimi

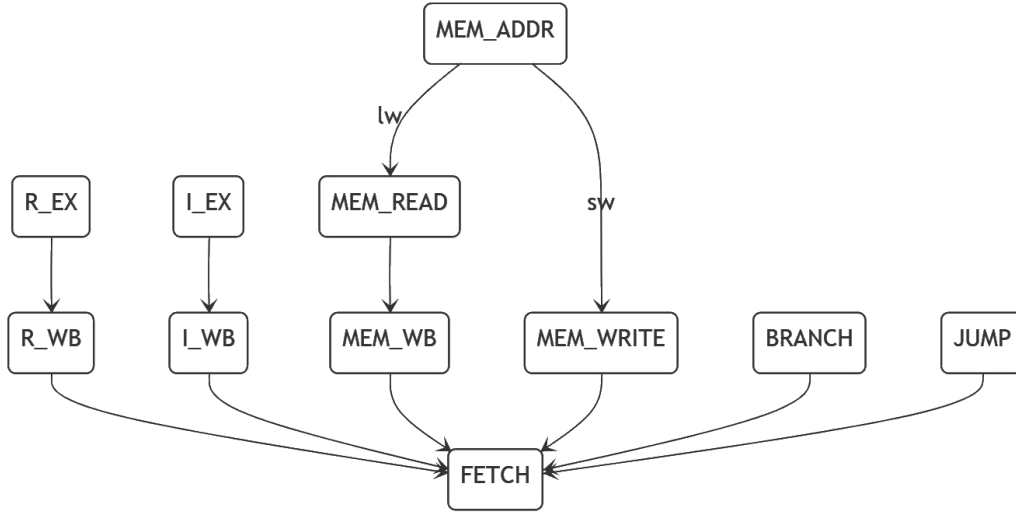
İki süreçli FSM: (1) senkron durum register'ı saat kenarında durum geçişini yapar; (2) bileşimsel süreç çıkış ve sonraki durumu üretir. Bileşimsel süreç başında tüm çıkışlara '0' atanarak latch oluşumu engellenir.

### 4.1 Durum geçiş diyagramı

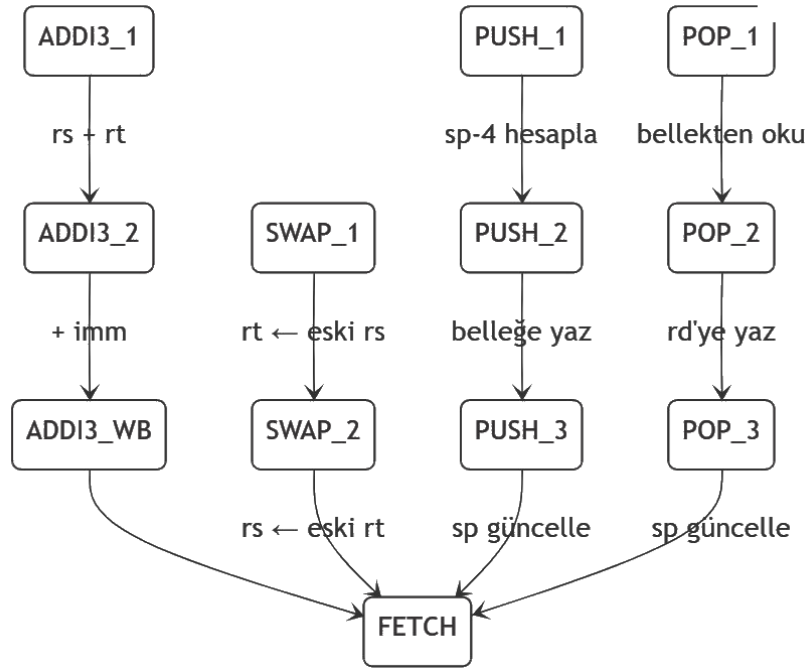
FSM her komutta S\_FETCH durumundan başlar ve S\_DECODE durumunda komut tipine göre dallanır. Aşağıdaki üç diyagram sırasıyla; getirme/çözme aşamasındaki dallanmayı, standart komutların yürütme-bellek-geri yazma yollarının S\_FETCH'e dönüşünü ve yeni komutların (addi3, swap, push, pop) çok adımlı geçişlerini göstermektedir.



Şekil 4.1 — Getirme ve çözme aşaması. S\_FETCH → S\_DECODE geçişinin ardından komut tipine (R-type, I-type, lw/sw, beq/bne/bgt, j, addi3, swap, push, pop) göre ilgili yürütme durumuna dallanılır.



Şekil 4.2 — Standart komutların tamamlanması. R-type ve I-type yürütme durumları geri yazma (WB) sonrası; lw/sw bellek erişim durumları, branch ve jump ise doğrudan S\_FETCH durumuna döner. MEM\_ADDR durumu komuta göre lw için MEM\_READ, sw için MEM\_WRITE yoluna ayrılır.



Şekil 4.3 — Yeni komutların durum geçişleri. addi3 iki ALU çevrimi (EX1→EX2) sonrası geri yazar; swap iki ardışık yazma çevrimiyle (W1→W2) takası tamamlar; push ve pop ise adres/bellek/yığın-işaretçisi adımlarını izleyerek S\_FETCH durumuna döner.

## 4.2 FSM durumları (23 adet)

| Durum      | Aşama | Açıklama                                              |
|------------|-------|-------------------------------------------------------|
| S_FETCH    | IF    | Komut bellekten okunur, IR'a kilitlenir. ALU: PC+4.   |
| S_DECODE   | ID    | Registerlar okunur (A,B). Dallanma hedefi hesaplanır. |
| S_RTYPE_EX | EX    | ALU: A op B → ALUOut. funct'tan op kodu çözülür.      |
| S_RTYPE_WB | WB    | rd ← ALUOut.                                          |

| Durum       | Aşama | Açıklama                                                                          |
|-------------|-------|-----------------------------------------------------------------------------------|
| S_ITYPE_EX  | EX    | ALU: $A \text{ op } \text{signext16} \rightarrow \text{ALUOut}$ .                 |
| S_ITYPE_WB  | WB    | $rt \leftarrow \text{ALUOut}$ .                                                   |
| S_MEM_ADDR  | EX    | Etkin adres = $A + \text{signext16} \rightarrow \text{ALUOut}$ .                  |
| S_MEM_READ  | MEM   | $\text{MDR} \leftarrow \text{MEM}[\text{ALUOut}]$ .                               |
| S_MEM_WB    | WB    | $rt \leftarrow \text{MDR}$ .                                                      |
| S_MEM_WRITE | MEM   | $\text{MEM}[\text{ALUOut}] \leftarrow B$ .                                        |
| S_BRANCH    | EX    | ALU: $A - B$ (zero/neg). Koşul sağlanırsa $\text{PC} \leftarrow \text{hedef}$ .   |
| S_JUMP      | –     | $\text{PC} \leftarrow \text{jump\_addr}$ .                                        |
| S_ADDI3_EX1 | EX    | ALU: $A + B \rightarrow \text{ALUOut}$ ( $rs + rt$ ).                             |
| S_ADDI3_EX2 | EX    | ALU: $\text{ALUOut} + \text{signext11} \rightarrow \text{ALUOut}$ (geri besleme). |
| S_ADDI3_WB  | WB    | $rd \leftarrow \text{ALUOut}$ .                                                   |
| S_SWAP_W1   | WB    | $rt \leftarrow A$ (eski $rs$ değeri).                                             |
| S_SWAP_W2   | WB    | $rs \leftarrow B$ (eski $rt$ değeri).                                             |
| S_PUSH_ADDR | EX    | ALU: $\$sp - 4 \rightarrow \text{ALUOut}$ .                                       |
| S_PUSH_MEM  | MEM   | $\text{MEM}[\text{ALUOut}] \leftarrow B$ ( $rs$ ).                                |
| S_PUSH_SP   | WB    | $\$sp \leftarrow \text{ALUOut}$ .                                                 |
| S_POP_READ  | MEM   | $\text{MDR} \leftarrow \text{MEM}[\$sp]$ ; $\text{ALUOut} = \$sp + 4$ .           |
| S_POP_WB    | WB    | $rt \leftarrow \text{MDR}$ .                                                      |
| S_POP_SP    | WB    | $\$sp \leftarrow \text{ALUOut}$ .                                                 |

### 4.3 Komut başına çevrim sayıları

| Komut tipi              | Örnekler                           | Çevrim |
|-------------------------|------------------------------------|--------|
| R-type (standart + mul) | add, sub, and, or, xor, slt, mul   | 4      |
| I-type aritmetik        | addi, slti, andi, ori, xori, loadi | 4      |
| LW                      | lw rt, offset(rs)                  | 5      |
| SW                      | sw rt, offset(rs)                  | 4      |
| Branch                  | beq, bne, bgt                      | 3      |
| Jump                    | j                                  | 3      |
| ADDI3                   | addi3 rd,rs,rt,imm                 | 5      |
| SWAP                    | swap rs,rt                         | 4      |
| PUSH                    | push rs                            | 5      |
| POP                     | pop rd                             | 5      |

### 4.4 Dallanma koşulları



| Komut | pc_write koşulu          | Açıklama                                  |
|-------|--------------------------|-------------------------------------------|
| beq   | zero='1'                 | A-B = 0 ise dallanır                      |
| bne   | NOT zero                 | A-B ≠ 0 ise dallanır                      |
| bgt   | (NOT zero) AND (NOT neg) | A-B > 0 ise dallanır; eşitlikte dallanmaz |

## 5. Komut Kümesi Tasarımı

### 5.1 Standart komutlar

| Komut | Tip | Opcode/Funct | İşlem                            |
|-------|-----|--------------|----------------------------------|
| add   | R   | funct=0x20   | rd = rs + rt                     |
| sub   | R   | funct=0x22   | rd = rs - rt                     |
| and   | R   | funct=0x24   | rd = rs AND rt                   |
| or    | R   | funct=0x25   | rd = rs OR rt                    |
| xor   | R   | funct=0x26   | rd = rs XOR rt                   |
| slt   | R   | funct=0x2a   | rd = (rs < rt) ? 1 : 0           |
| addi  | I   | op=0x08      | rt = rs + signext(imm)           |
| slti  | I   | op=0x0a      | rt = (rs < signext(imm)) ? 1 : 0 |
| andi  | I   | op=0x0c      | rt = rs AND signext(imm)         |
| ori   | I   | op=0x0d      | rt = rs OR signext(imm)          |
| xori  | I   | op=0x0e      | rt = rs XOR signext(imm)         |
| lw    | I   | op=0x23      | rt = MEM[rs + signext(imm)]      |
| sw    | I   | op=0x2b      | MEM[rs + signext(imm)] = rt      |
| beq   | I   | op=0x04      | rs==rt ise dallan                |
| bne   | I   | op=0x05      | rs!=rt ise dallan                |
| j     | J   | op=0x02      | PC ← PC[31:28]   imm26<<2        |

### 5.2 Yeni komutlar (7 adet)

#### MUL — Çarpma

R-type · opcode=0x00 · funct=0x2c · sözdizimi: **mul rd, rs, rt**

rd = rs × rt (signed, alt 32-bit sonuç). Standart R-type ardışık düzenini izler (4 çevrim). S\_RTYPE\_EX durumunda `alu_ctrl="1000"` ile ALU çarpma yapar. Sentezde çok çevrimli bir çarpıcıya genişletilebilir.

#### SWAP — Register takası

R-type · opcode=0x00 · funct=0x2d · sözdizimi: **swap rs, rt**

rs ↔ rt. Register file'da okuma, yazmadan önce eski değeri döndürür. Bu özellik kullanılarak iki ardışık yazma çevrimiyle takas gerçekleştirilir: S\_SWAP\_W1'de rt ← A (eski rs), S\_SWAP\_W2'de rs ← B (eski rt). (4 çevrim)

#### LOADI — Immediate yükleme

I-type · opcode=0x11 · rs=0 · rt=rd · imm(16) · sözdizimi: `loadi rd, imm`

rd = imm16. ALU:  $0 + \text{imm} = \text{imm}$  (\$0 register'ı kaynak olarak kullanılır). I-type ardışık düzenini izler (4 çevrim). 16-bit signed veya unsigned desteklenir.

### BGT — Büyükse dallan

I-type · opcode=0x12 · rs · rt · offset(16) · sözdizimi: `bgt rs, rt, label`

rs > rt ise dallanır. S\_BRANCH durumunda A-B hesaplanır; zero=0 VE neg=0 koşulu rs>rt anlamına gelir. Eşitlikte dallanmaz. (3 çevrim)

### ADDI3 — Üç operandlı toplama

Special · opcode=0x10 · rs(5) | rt(5) | rd(5) | imm(11) · sözdizimi: `addi3 rd,rs,rt,imm`

rd = rs + rt + imm11 (signed). İki ALU çevrimi gerektirir: S\_ADDI3\_EX1'de rs+rt → ALUOut; S\_ADDI3\_EX2'de `alu_src_a="10"` ile ALUOut geri beslenip ALUOut + signext11 → ALUOut. (5 çevrim — tasarımın özgün özelliği)

### PUSH — Yığına it

I-type · opcode=0x13 · rs · rt=0 · imm=0 · sözdizimi: `push rs`

\$sp -= 4; MEM[\$sp] = rs. reg1\_sel=1 ile A=\$sp, reg2\_sel=1 ile B=rs seçilir. Üç FSM durumu: S\_PUSH\_ADDR → S\_PUSH\_MEM → S\_PUSH\_SP. (5 çevrim)

### POP — Yığından çek

I-type · opcode=0x14 · rs=0 · rt=rd · imm=0 · sözdizimi: `pop rd`

rd = MEM[\$sp]; \$sp += 4. `ior_d="10"` ile bellek adresi olarak A (\$sp) kullanılır. Üç FSM durumu: S\_POP\_READ → S\_POP\_WB → S\_POP\_SP. LIFO anlamı korunur. (5 çevrim)

## 6. Assembler Tasarımı — assembler/assembler.py

Python 3 ile yazılmış iki geçişli assembler 23 komutu destekler. Çıktı, ModelSim `$readmemh` biçiminde bir `.hex` dosyasıdır (her satır 32-bit bir sözcük).

### 6.1 İki geçişli derleme akışı

```
Kaynak (.asm)
|
▼ Geçiş 1 : etiketleri tara, label_table[label]=byte_addr
|           her komut için byte_addr += 4
▼ Geçiş 2 : her komutu 32-bit makine koduna çevir
|           branch/jump etiketlerini label_table'dan çöz
▼ write_hex() → program.hex ($readmemh biçimi)
```

### 6.2 Kodlama biçimleri

| Biçim  | Kodlama                                              |
|--------|------------------------------------------------------|
| R-type | 000000   rs(5)   rt(5)   rd(5)   shamt(5)   funct(6) |
| I-type | opcode(6)   rs(5)   rt(5)   imm(16, signed)          |
| J-type | opcode(6)   address(26)                              |
| ADDI3  | 010000   rs(5)   rt(5)   rd(5)   imm(11, signed)     |
| LOADI  | 010001   00000   rt(5)   imm(16)                     |

| Biçim | Kodlama                                   |
|-------|-------------------------------------------|
| PUSH  | 010011   rs(5)   00000   0000000000000000 |
| POP   | 010100   00000   rt(5)   0000000000000000 |

### 6.3 Dallanma offset hesabı

$\text{offset} = (\text{target\_byte\_addr} - (\text{PC} + 4)) \gg 2$ . İleri ve geri dallanma desteklenir; aralık aşımında SyntaxError fırlatılır.

### 6.4 ISA tek doğru kaynak

R\_FUNCT, I\_OPCODE ve ADDI3\_OP sözlükleri hem assembler hem VHDL için tek doğru kaynaktır. `control_unit.vhd` ve `alu_control.vhd` aynı opcode sabitlerini kullanır; uyumsuzluklar doğrulama testlerinde yakalanır.

## 7. Test ve Doğrulama

Hiyerarşik bir test stratejisi izlenmiştir: önce her modül bağımsız birim testine tabi tutulmuş, ardından tam sistem bütünleşme testiyle doğrulanmıştır. Tüm testbench'ler kendi kendini denetler; beklenen değerlerle assert karşılaştırması yapar. Testler ModelSim Intel FPGA Edition 2020.1 ortamında 03 Haziran 2026'da çalıştırılmıştır.

### 7.1 ALU birim testi (tb\_alu)

Amaç: ADD, SUB, AND, OR, XOR, SLT ve MUL işlemleri ile zero bayrağı davranışını doğrulamak. ALU bileşimsel olduğundan saat gerekmez; her test vektörü 10 ns aralıkla uygulanır.

| Test      | a  | b  | Beklenen |
|-----------|----|----|----------|
| ADD +5+7  | 5  | 7  | 12       |
| ADD -3+10 | -3 | 10 | 7        |
| SUB 20-8  | 20 | 8  | 12       |
| SUB 5-9   | 5  | 9  | -4       |
| AND 12&10 | 12 | 10 | 8        |
| OR 12 10  | 12 | 10 | 14       |
| XOR 12^10 | 12 | 10 | 6        |
| SLT 5<7   | 5  | 7  | 1        |
| SLT 7<5   | 7  | 5  | 0        |
| SLT -2<3  | -2 | 3  | 1        |
| MUL 6×7   | 6  | 7  | 42       |
| MUL -4×5  | -4 | 5  | -20      |
| zero: 5-5 | 5  | 5  | zero='1' |
| zero: 5-3 | 5  | 3  | zero='0' |

| Test                                                                     | a | b | Beklenen |
|--------------------------------------------------------------------------|---|---|----------|
| ✓ Tüm ALU testleri geçti — Errors: 0, Warnings: 4 (metavalue, zararsız). |   |   |          |

```

Transcript
VSIM 120> vcom alu.vhd
# Model Technology ModelSim - Intel FPGA Edition vcom 2020.1 Compiler 2020.02 Feb 28 2020
# Start time: 14:26:06 on Jun 03,2026
# vcom -reportprogress 300 alu.vhd
# -- Loading package STANDARD
# -- Loading package TEXTIO
# -- Loading package std_logic_1164
# -- Loading package NUMERIC_STD
# -- Compiling entity alu
# -- Compiling architecture Behavioral of alu
# End time: 14:26:06 on Jun 03,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
VSIM 121> vcom tb_alu.vhd
# Model Technology ModelSim - Intel FPGA Edition vcom 2020.1 Compiler 2020.02 Feb 28 2020
# Start time: 14:26:06 on Jun 03,2026
# vcom -reportprogress 300 tb_alu.vhd
# -- Loading package STANDARD
# -- Loading package TEXTIO
# -- Loading package std_logic_1164
# -- Loading package NUMERIC_STD
# -- Compiling entity tb_alu
# -- Compiling architecture sim of tb_alu
# -- Loading entity alu
# End time: 14:26:06 on Jun 03,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
VSIM 122> vsim work.tb_alu
# End time: 14:26:07 on Jun 03,2026, Elapsed time: 0:22:25
# Errors: 0, Warnings: 4
# vsim work.tb_alu
# Start time: 14:26:07 on Jun 03,2026
# Loading std.standard
# Loading std.textio(body)
# Loading ieee.std_logic_1164(body)
# Loading ieee.numeric_std(body)
# Loading work.tb_alu(sim)
# Loading work.alu(behavioral)
VSIM 123> add wave *
VSIM 124> run -all
# ** Note: === ALU TESTLERI BASLIYOR ===
# Time: 0 ps Iteration: 0 Instance: /tb_alu
# ** Note: === TUM ALU TESTLERI GECTI ===
# Time: 140 ns Iteration: 0 Instance: /tb_alu
VSIM 125> wave zoom full
# {0 ps} {147 ns}
VSIM 126>

```

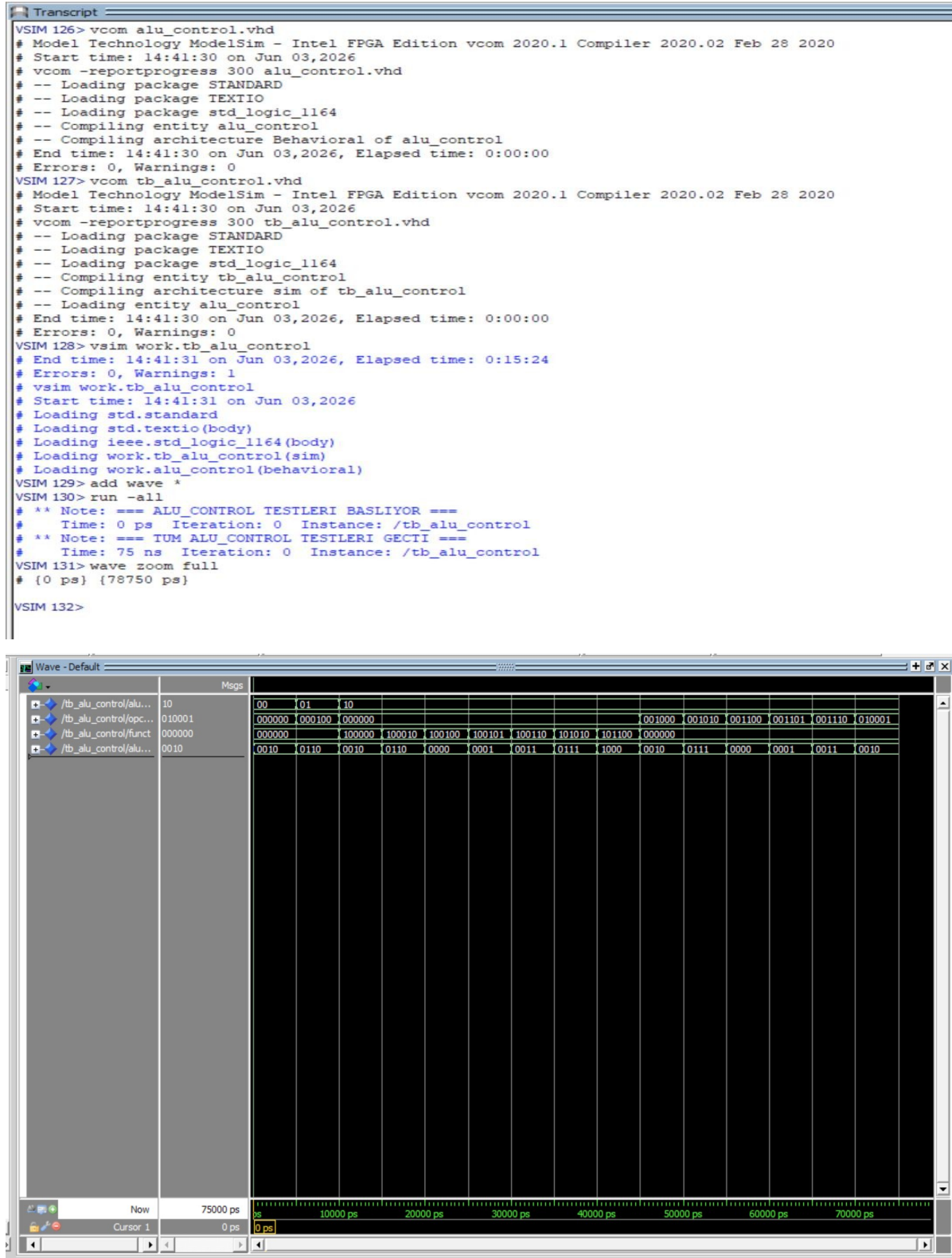


Şekil 7.1 — *tb\_alu* ALU birim testi dalga biçimi. *alu\_ctrl* girişi sırasıyla *ADD(0010)*, *SUB(0110)*, *AND(0000)*, *OR(0001)*, *XOR(0011)*, *SLT(0111)* ve *MUL(1000)* kodlarıyla sürülmüş; her değişimde bileşimsel devre yayılım gecikmesi içinde doğru sonucu üretmiştir. zero bayrağı  $5-5=0$  durumunda '1', diğer durumlarda '0' olmuştur. Toplam simülasyon süresi: 140 ns.

## 7.2 ALU denetim birimi testi (tb\_alu\_control)

Amaç: *alu\_op* / *opcode* / *funct* kombinasyonlarının doğru 4-bit *alu\_ctrl* ürettiğini doğrulamak. 15 kombinasyon test edilmiştir: zorlanmış *ADD/SUB* *alu\_op* + 7 R-type *funct* + 6 I-type *opcode*. Her test 5 ns aralıkla uygulanır.

✓ Tüm ALU\_CONTROL testleri geçti — Errors: 0, Warnings: 1 (zararsız).



Şekil 7.2 — `tb_alu_control` dalga biçimi. `opcode` ve `funct` girişleri farklı kombinasyonlarla sürülmüş; `alu_op="10"` modunda `R-type` komutlar için `funct` alanına, `I-type` komutlar için `opcode` alanına göre doğru `alu_ctrl` değerinin üretildiği gözlemlenmiştir. 15 `assert`'in tamamı geçmiş, simülasyon 75 ns içinde tamamlanmıştır.

### 7.3 I-type komut testi (`tb_itype`)

Amaç: diğer testbench'lerde doğrudan kullanılmayan `slti`, `andi`, `ori`, `xori` komutlarını bağımsız doğrulamak. `test_itype.hex` yüklenerek 200 saat çevriminde 10 register doğrulanır.



| Komut               | İşlem         | Beklenen |
|---------------------|---------------|----------|
| loadi \$t0, 12      | \$t0 = 12     | \$t0=12  |
| slti \$t1, \$t0, 20 | 12 < 20 → 1   | \$t1=1   |
| slti \$t2, \$t0, 5  | 12 < 5 → 0    | \$t2=0   |
| slti \$t3, \$t0, 12 | 12 < 12 → 0   | \$t3=0   |
| andi \$t4, \$t0, 10 | 12 AND 10 = 8 | \$t4=8   |
| ori \$t5, \$t0, 3   | 12 OR 3 = 15  | \$t5=15  |
| xori \$t6, \$t0, 10 | 12 XOR 10 = 6 | \$t6=6   |
| loadi \$t7, -3      | \$t7 = -3     | \$t7=-3  |
| slti \$s0, \$t7, 0  | -3 < 0 → 1    | \$s0=1   |
| slti \$s1, \$t7, -5 | -3 < -5 → 0   | \$s1=0   |

✓ Tüm ITYPE testleri geçti — Errors: 0, Warnings: 1 (zararsız).

```

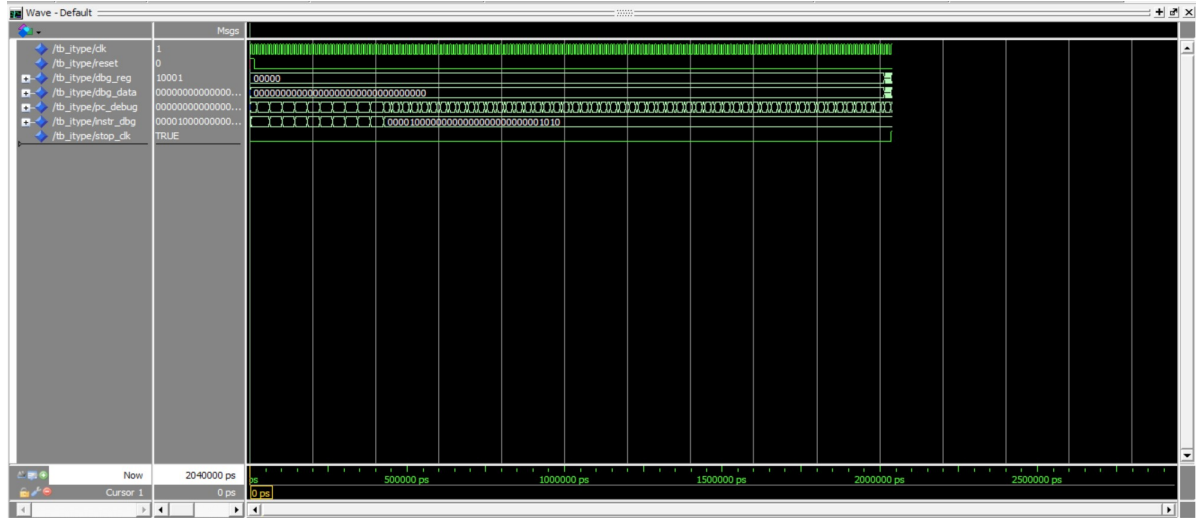
Transcript
VSIM 132> vcom -2008 memory.vhd
# Model Technology ModelSim - Intel FPGA Edition vcom 2020.1 Compiler 2020.02 Feb 28 2020
# Start time: 14:49:33 on Jun 03,2026
# vcom -reportprogress 300 -2008 memory.vhd
# -- Loading package STANDARD
# -- Loading package TEXTIO
# -- Loading package std_logic_1164
# -- Loading package NUMERIC_STD
# -- Compiling entity memory
# -- Compiling architecture Behavioral of memory
# End time: 14:49:33 on Jun 03,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
VSIM 133> vcom datapath.vhd
# Model Technology ModelSim - Intel FPGA Edition vcom 2020.1 Compiler 2020.02 Feb 28 2020
# Start time: 14:49:33 on Jun 03,2026
# vcom -reportprogress 300 datapath.vhd
# -- Loading package STANDARD
# -- Loading package TEXTIO
# -- Loading package std_logic_1164
# -- Loading package NUMERIC_STD
# -- Compiling entity datapath
# -- Compiling architecture Behavioral of datapath
# -- Loading entity pc
# -- Loading entity memory
# -- Loading entity instruction_register
# -- Loading entity simple_register
# -- Loading entity register_file
# -- Loading entity alu
# End time: 14:49:33 on Jun 03,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
VSIM 134> vcom cpu.vhd
# Model Technology ModelSim - Intel FPGA Edition vcom 2020.1 Compiler 2020.02 Feb 28 2020
# Start time: 14:49:33 on Jun 03,2026
# vcom -reportprogress 300 cpu.vhd
# -- Loading package STANDARD
# -- Loading package TEXTIO
# -- Loading package std_logic_1164
# -- Compiling entity cpu
# -- Compiling architecture Structural of cpu
# -- Loading entity control_unit
# -- Loading entity alu_control
# -- Loading package NUMERIC_STD
# -- Loading entity datapath
# End time: 14:49:33 on Jun 03,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0

```

```

VSIM 135> vcom tb_itype.vhd
# Model Technology ModelSim - Intel FPGA Edition vcom 2020.1 Compiler 2020.02 Feb 28 2020
# Start time: 14:49:33 on Jun 03,2026
# vcom -reportprogress 300 tb_itype.vhd
# -- Loading package STANDARD
# -- Loading package TEXTIO
# -- Loading package std_logic_1164
# -- Loading package NUMERIC_STD
# -- Compiling entity tb_itype
# -- Compiling architecture sim of tb_itype
# -- Loading entity cpu
# End time: 14:49:33 on Jun 03,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
VSIM 136> vsim work.tb_itype
# End time: 14:49:34 on Jun 03,2026, Elapsed time: 0:08:03
# Errors: 0, Warnings: 1
# vsim work.tb_itype
# Start time: 14:49:34 on Jun 03,2026
# Loading std.standard
# Loading std.textio(body)
# Loading ieee.std_logic_1164(body)
# Loading ieee.numeric_std(body)
# Loading work.tb_itype(sim)
# Loading work.cpu(structural)
# Loading work.control_unit(behavioral)
# Loading work.alu_control(behavioral)
# Loading work.datapath(behavioral)
# Loading work.pc(behavioral)
# Loading work.memory(behavioral)
# Loading work.instruction_register(behavioral)
# Loading work.simple_register(behavioral)
# Loading work.register_file(behavioral)
# Loading work.alu(behavioral)
VSIM 137> add wave *
VSIM 138> run -all
# ** Note: === ITYPE (slti/andi/ori/xori) TESTİ BASLIYOR ===
# Time: 0 ps Iteration: 0 Instance: /tb_itype
# ** Warning: NUMERIC_STD.TO_INTEGER: metavalue detected, returning 0
# Time: 0 ps Iteration: 0 Instance: /tb_itype/DUT/DP/RF_INST
# ** Warning: NUMERIC_STD.TO_INTEGER: metavalue detected, returning 0
# Time: 0 ps Iteration: 0 Instance: /tb_itype/DUT/DP/RF_INST
# ** Warning: NUMERIC_STD.TO_INTEGER: metavalue detected, returning 0
# Time: 0 ps Iteration: 1 Instance: /tb_itype/DUT/DP/MEM_INST
# ** Note: === TUM ITYPE TESTLERİ GEÇTİ (Error yoksa) ===
# Time: 2036 ns Iteration: 0 Instance: /tb_itype
VSIM 139> wave zoom full
# {0 ps} {2142 ns}
VSIM 140>

```



Şekil 7.3 — *tb\_itype* dalga biçimi. *pc\_debug* sinyalinin her komutta 4 bayt ilerlediği; *dbg\_data* çıkışının test programı tamamlandığında 10 register için beklenen değerleri tuttuğu görülmektedir. Negatif immediate ( $\$t7=-3$ ) ve signed karşılaştırma ( $\$s0=1: -3<0$ ,  $\$s1=0: -3<-5$  değil) doğru sonuçlanmıştır. Simülasyon süresi: 2036 ns.

## 7.4 Yığın mekanizması testi (*tb\_stack*)

Amaç: push, pop ve swap komutlarının LIFO anlamını ve \$sp güncellemesini doğrulamak. test\_stack.hex yüklenerek 100 saat çevriminde 5 register doğrulanır.



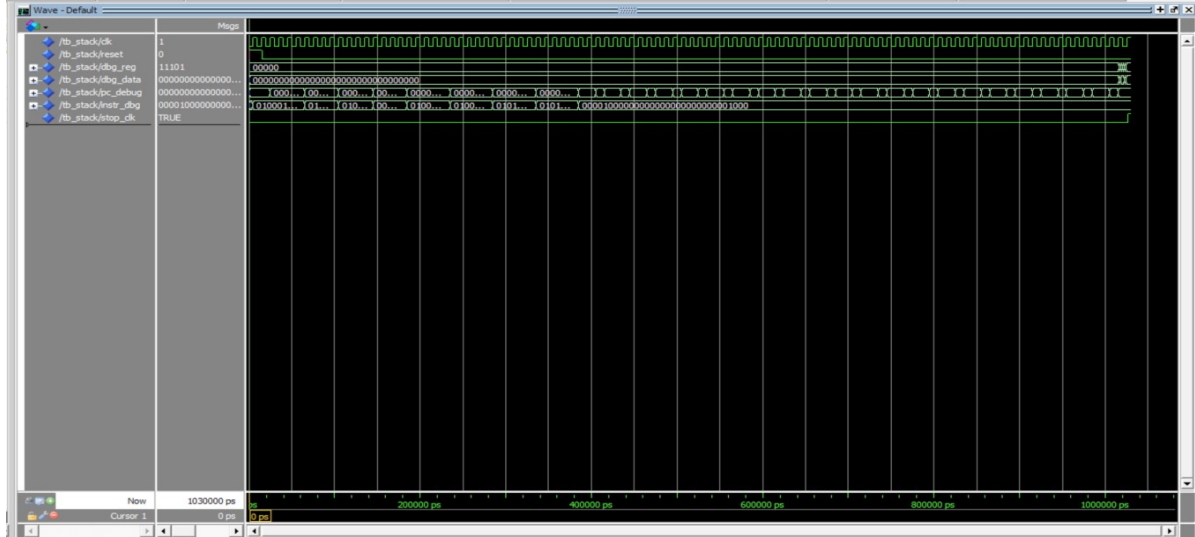
```

$sp = 400
loadi $t0, 11 | loadi $t1, 22
swap $t0, $t1 -> $t0=22, $t1=11
push $t0 (22) -> MEM[396]=22, $sp=396
push $t1 (11) -> MEM[392]=11, $sp=392
pop $t2 -> $t2=11, $sp=396 (LIFO: son push ilk çıkar)
pop $t3 -> $t3=22, $sp=400

```

| Register  | Beklenen | Açıklama                   |
|-----------|----------|----------------------------|
| \$t0 (8)  | 22       | swap sonrası               |
| \$t1 (9)  | 11       | swap sonrası               |
| \$t2 (10) | 11       | LIFO pop sırası            |
| \$t3 (11) | 22       | LIFO pop sırası            |
| \$sp (29) | 400      | Başlangıç değerine dönmeli |

✓ Tüm STACK testleri geçti — Errors: 0, Warnings: 4 (zararsız).



```

Transcript
VSIM 140> vcom tb_stack.vhd
# Model Technology ModelSim - Intel FPGA Edition vcom 2020.1 Compiler 2020.02 Feb 28 2020
# Start time: 14:58:00 on Jun 03,2026
# vcom -reportprogress 300 tb_stack.vhd
# -- Loading package STANDARD
# -- Loading package TEXTIO
# -- Loading package std_logic_1164
# -- Loading package NUMERIC_STD
# -- Compiling entity tb_stack
# -- Compiling architecture sim of tb_stack
# -- Loading entity cpu
# End time: 14:58:00 on Jun 03,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
VSIM 141> vsim work.tb_stack
# End time: 14:58:01 on Jun 03,2026, Elapsed time: 0:08:27
# Errors: 0, Warnings: 4
# vsim work.tb_stack
# Start time: 14:58:01 on Jun 03,2026
# Loading std.standard
# Loading std.textio(body)
# Loading ieee.std_logic_1164(body)
# Loading ieee.numeric_std(body)
# Loading work.tb_stack(sim)
# Loading work.cpu(structural)
# Loading work.control_unit(behavioral)
# Loading work.alu_control(behavioral)
# Loading work.datapath(behavioral)
# Loading work.pc(behavioral)
# Loading work.memory(behavioral)
# Loading work.instruction_register(behavioral)
# Loading work.simple_register(behavioral)
# Loading work.register_file(behavioral)
# Loading work.alu(behavioral)
VSIM 142> add wave *
VSIM 143> run -all
# ** Note: === STACK (push/pop/swap) TESTİ BASLIYOR ===
# Time: 0 ps Iteration: 0 Instance: /tb_stack
# ** Warning: NUMERIC_STD.TO_INTEGER: metavalue detected, returning 0
# Time: 0 ps Iteration: 0 Instance: /tb_stack/DUT/DP/RF_INST
# ** Warning: NUMERIC_STD.TO_INTEGER: metavalue detected, returning 0
# Time: 0 ps Iteration: 0 Instance: /tb_stack/DUT/DP/RF_INST
# ** Warning: NUMERIC_STD.TO_INTEGER: metavalue detected, returning 0
# Time: 0 ps Iteration: 1 Instance: /tb_stack/DUT/DP/MEM_INST
# ** Note: === TUM STACK TESTLERİ GEÇTİ ===
# Time: 1026 ns Iteration: 0 Instance: /tb_stack
VSIM 144> wave zoom full
# {0 ps} {1081500 ps}
VSIM 145>

```

Şekil 7.4 — *tb\_stack* dalga biçimi. *instr\_dbg* sinyalinde komutların sırayla *PUSH* ve *POP* opcode'larını taşıdığı görülmüştür. *dbg\_data* çıkışı *\$sp* register'ını izleyerek her *PUSH*'ta 4 azaldığını, her *POP*'ta 4 arttığını ortaya koymuştur. Program sonunda *\$sp=400* ile yığının tam dengeye geldiği ve *LIFO* sıralamasının (*\$t2=11*, *\$t3=22*) doğru olduğu doğrulanmıştır. Simülasyon süresi: 1026 ns.

## 7.5 Sınır durum testi (tb\_boundary)

Amaç: 32-bit signed overflow, bellek sınır erişimi (son sözcük) ve dallanma offset sınırları gibi sınır değerlerini test etmek.

| Register | Beklenen      | Açıklama                           |
|----------|---------------|------------------------------------|
| \$t0     | 30.000        | Temel değer                        |
| \$t1     | 900.000.000   | $30000 \times 30000$ — MUL ile     |
| \$t2     | 1.800.000.000 | $\$t1 + \$t1$ — henüz overflow yok |

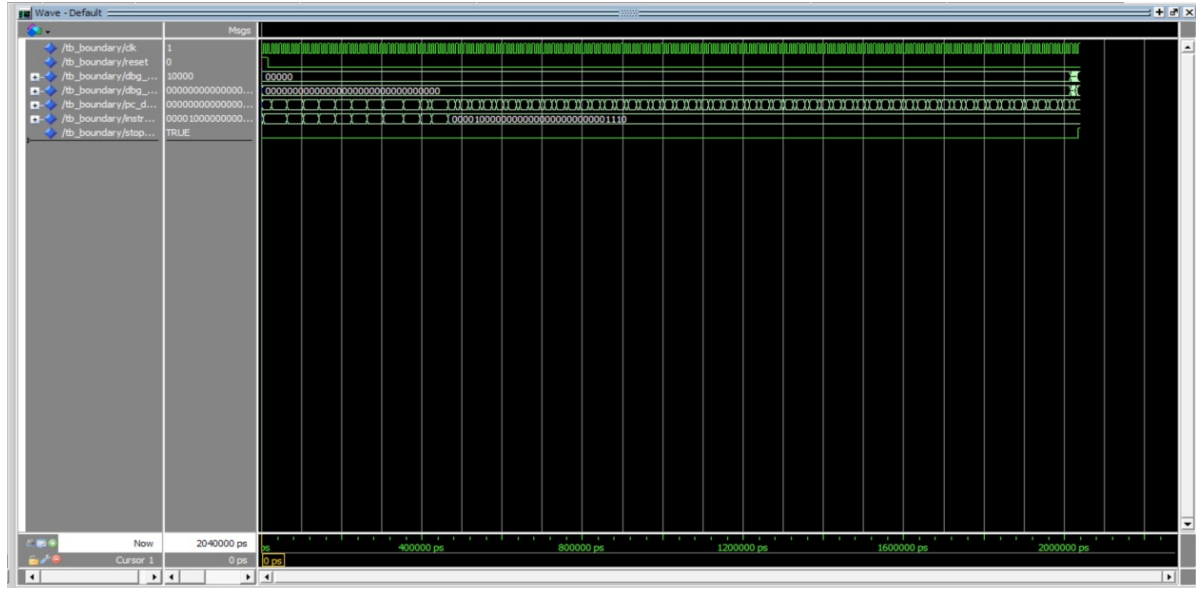
| Register | Beklenen       | Açıklama                           |
|----------|----------------|------------------------------------|
| \$t3     | -1.594.967.296 | 32-bit wrap-around (overflow)      |
| \$t4     | 1020           | Son geçerli bayt adresi (word 255) |
| \$t5     | 12345          | Belleğe yazılan değer              |
| \$t6     | 12345          | MEM[1020] geri okundu — tutarlı    |
| \$t7     | 0              | BEQ koşuluyla atlanan değer        |
| \$s0     | 7              | Dallanma sonrası yüklenen değer    |

✓ Tüm BOUNDARY testleri geçti — Errors: 0, Warnings: 4 (zararsız).

```

Transcript
VSIM 145> vcom tb_boundary.vhd
# Model Technology ModelSim - Intel FPGA Edition vcom 2020.1 Compiler 2020.02 Feb 28 2020
# Start time: 15:04:51 on Jun 03,2026
# vcom -reportprogress 300 tb_boundary.vhd
# -- Loading package STANDARD
# -- Loading package TEXTIO
# -- Loading package std_logic_1164
# -- Loading package NUMERIC_STD
# -- Compiling entity tb_boundary
# -- Compiling architecture sim of tb_boundary
# -- Loading entity cpu
# End time: 15:04:51 on Jun 03,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
VSIM 146> vsim work.tb_boundary
# End time: 15:04:51 on Jun 03,2026, Elapsed time: 0:06:50
# Errors: 0, Warnings: 4
# vsim work.tb_boundary
# Start time: 15:04:51 on Jun 03,2026
# Loading std.standard
# Loading std.textio(body)
# Loading ieee.std_logic_1164(body)
# Loading ieee.numeric_std(body)
# Loading work.tb_boundary(sim)
# Loading work.cpu(structural)
# Loading work.control_unit(behavioral)
# Loading work.alu_control(behavioral)
# Loading work.datapath(behavioral)
# Loading work.pc(behavioral)
# Loading work.memory(behavioral)
# Loading work.instruction_register(behavioral)
# Loading work.simple_register(behavioral)
# Loading work.register_file(behavioral)
# Loading work.alu(behavioral)
VSIM 147> add wave *
VSIM 148> run -all
# ** Note: === BOUNDARY (sinir durumları) TESTİ BASLIYOR ===
# Time: 0 ps Iteration: 0 Instance: /tb_boundary
# ** Warning: NUMERIC_STD.TO_INTEGER: metavalue detected, returning 0
# Time: 0 ps Iteration: 0 Instance: /tb_boundary/DUT/DP/RF_INST
# ** Warning: NUMERIC_STD.TO_INTEGER: metavalue detected, returning 0
# Time: 0 ps Iteration: 0 Instance: /tb_boundary/DUT/DP/RF_INST
# ** Warning: NUMERIC_STD.TO_INTEGER: metavalue detected, returning 0
# Time: 0 ps Iteration: 1 Instance: /tb_boundary/DUT/DP/MEM_INST
# ** Note: === TUM BOUNDARY TESTLERİ GEÇTİ (Error yoksa) ===
# Time: 2034 ns Iteration: 0 Instance: /tb_boundary
VSIM 149> wave zoom full
# {0 ps} {2142 ns}
VSIM 150>

```



Şekil 7.5 — *tb\_boundary* dalga biçimi. *dbg\_data* çıkışı 32-bit aritmetiğin wrap-around davranışını (*\$t3* hesabında negatif sonuç) net biçimde göstermektedir. Bellek sınır erişimi (address=1020, word 255) başarılı okuma/yazma yapmıştır. BEQ tabanlı ileri dallanma, PC'nin doğru hedefe atlayıp atlamadığını doğrulamıştır. Simülasyon süresi: 2034 ns.

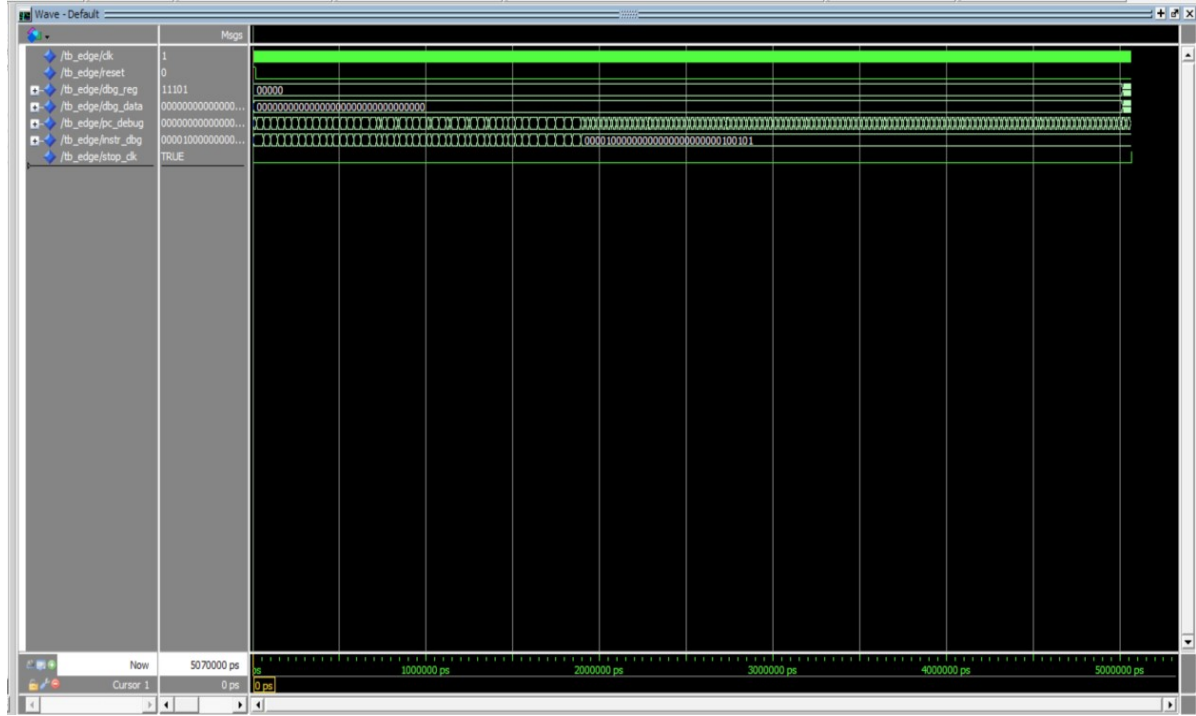
## 7.6 Uç durum testi (tb\_edge)

Amaç: signed sayılar, \$0 yazma koruması, signed karşılaştırma, BGT eşitlik durumu, geri dallanma sayacı ve iç içe push/pop gibi uç durumları test etmek. 500 saat çevriminde 25 register doğrulanır.

| Register   | Beklenen | Açıklama                                |
|------------|----------|-----------------------------------------|
| \$zero (0) | 0        | \$0 yazma koruması: değer değişmemeli   |
| \$t1 (9)   | -5       | loadi -5 (negatif immediate)            |
| \$t2 (10)  | 1        | slt: $-5 < 0 \rightarrow 1$ (signed)    |
| \$t3 (11)  | 0        | slt: $0 < -5 \rightarrow 0$             |
| \$t6 (14)  | -7       | sub: $3 - 10 = -7$                      |
| \$s0 (16)  | -15      | mul: $-5 \times 3 = -15$ (signed)       |
| \$s1 (17)  | -1       | loadi -1 (= 0xFFFFFFFF)                 |
| \$s2 (18)  | 10       | and: $10 \text{ AND } 0xFFFFFFFF = 10$  |
| \$s3 (19)  | -11      | xor: $10 \text{ XOR } 0xFFFFFFFF = -11$ |
| \$s5 (21)  | 9        | addi3: $10 + 3 + (-4) = 9$              |
| \$s6 (22)  | 1        | bgt taken: $10 > 5$                     |
| \$s7 (23)  | 77       | bgt not taken: eşitlikte ( $5=5$ )      |
| \$t8 (24)  | 0        | Geri dallanma sayacı: $5 \rightarrow 0$ |
| \$t9 (25)  | 5        | Döngü 5 kez döndü                       |
| \$fp (30)  | 300      | Nested pop — LIFO 1. çıkış              |
| \$ra (31)  | 200      | Nested pop — LIFO 2. çıkış              |

| Register  | Beklenen | Açıklama                   |
|-----------|----------|----------------------------|
| \$gp (28) | 100      | Nested pop — LIFO 3. çıkış |
| \$sp (29) | 400      | Yığın tam dengeye döndü    |

✓ Tüm EDGE testleri geçti — Errors: 0, Warnings: 4 (zararsız).



```

Transcript
VSIM 150> vcom tb_edge.vhd
# Model Technology ModelSim - Intel FPGA Edition vcom 2020.1 Compiler 2020.02 Feb 28 2020
# Start time: 15:10:34 on Jun 03,2026
# vcom -reportprogress 300 tb_edge.vhd
# -- Loading package STANDARD
# -- Loading package TEXTIO
# -- Loading package std_logic_1164
# -- Loading package NUMERIC_STD
# -- Compiling entity tb_edge
# -- Compiling architecture sim of tb_edge
# -- Loading entity cpu
# End time: 15:10:34 on Jun 03,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
VSIM 151> vsim work.tb_edge
# End time: 15:10:35 on Jun 03,2026, Elapsed time: 0:05:44
# Errors: 0, Warnings: 4
# vsim work.tb_edge
# Start time: 15:10:35 on Jun 03,2026
# Loading std.standard
# Loading std.textio(body)
# Loading ieee.std_logic_1164(body)
# Loading ieee.numeric_std(body)
# Loading work.tb_edge(sim)
# Loading work.cpu(structural)
# Loading work.control_unit(behavioral)
# Loading work.alu_control(behavioral)
# Loading work.datapath(behavioral)
# Loading work.pc(behavioral)
# Loading work.memory(behavioral)
# Loading work.instruction_register(behavioral)
# Loading work.simple_register(behavioral)
# Loading work.register_file(behavioral)
# Loading work.alu(behavioral)
VSIM 152> add wave *
VSIM 153> run -all
# ** Note: === EDGE (uc durum) TESTİ BASLIYOR ===
# Time: 0 ps Iteration: 0 Instance: /tb_edge
# ** Warning: NUMERIC_STD.TO_INTEGER: metavalue detected, returning 0
# Time: 0 ps Iteration: 0 Instance: /tb_edge/DUT/DP/RF_INST
# ** Warning: NUMERIC_STD.TO_INTEGER: metavalue detected, returning 0
# Time: 0 ps Iteration: 0 Instance: /tb_edge/DUT/DP/RF_INST
# ** Warning: NUMERIC_STD.TO_INTEGER: metavalue detected, returning 0
# Time: 0 ps Iteration: 1 Instance: /tb_edge/DUT/DP/MEM_INST
# ** Note: === TUM EDGE TESTLERİ GEÇTİ (Error yoksa) ===
# Time: 5066 ns Iteration: 0 Instance: /tb_edge
VSIM 154> wave zoom full
# {0 ps} {5323500 ps}
VSIM 155>

```

Şekil 7.6 — *tb\_edge* dalga biçimi. Simülasyon boyunca \$zero register'ının 0 değerini koruduğu, negatif signed aritmetiğin doğru sonuçlar verdiği görülmüştür. BGT eşitlik durumunda (5=5) dallanmamış; geri dallanma, döngü sayacını tam olarak 5 iterasyonda sıfırlamıştır. Üç katmanlı iç içe push/pop LIFO sırasını koruyarak \$sp'yi başlangıç değerine (400) geri getirmiştir. Simülasyon süresi: 5066 ns.

## 7.7 Kapsamlı bütünleşme testi (tb\_master)

Amaç: tek bir programda 23 komutun tamamını gerçekçi bir veri işleme akışında kullanarak işlemcinin bütünleşik çalışmasını doğrulamak. Program; dizi kurma, toplam/maksimum hesaplayan döngü, aritmetik/mantık işlemler ve yığın testini kapsar. 600 saat çevriminde 18 register doğrulanır.

Dizi içeriği: [5, 8, 3, 10] → toplam = 26, maksimum = 10.

| Register  | Beklenen | Açıklama                         |
|-----------|----------|----------------------------------|
| \$t0 (8)  | 200      | Dizi taban adresi                |
| \$t1 (9)  | 3        | Son loadı değeri                 |
| \$t2 (10) | 16       | Döngü sayacı (4 eleman × 4 bayt) |
| \$t3 (11) | 16       | Döngü bitiş eşiği                |
| \$t4 (12) | 212      | Son dizi adresi                  |
| \$t5 (13) | 10       | Son okunan dizi elemanı          |



| Register  | Beklenen | Açıklama                   |
|-----------|----------|----------------------------|
| \$t6 (14) | 16       | sub: 26-10=16              |
| \$t7 (15) | 10       | and: 26 AND 10=10          |
| \$s0 (16) | 26       | Dizi toplamı               |
| \$s1 (17) | 10       | Dizi maksimumu             |
| \$s2 (18) | 26       | or: 26 OR 10=26            |
| \$s3 (19) | 16       | xor: 26 XOR 10=16          |
| \$s4 (20) | 1        | slt: 10<26 → 1             |
| \$s5 (21) | 30       | mul: 10 × 3 = 30           |
| \$s6 (22) | 40       | addi3: 26+10+4=40          |
| \$t8 (24) | 26       | swap sonrası (LIFO + swap) |
| \$t9 (25) | 10       | swap sonrası               |
| \$sp (29) | 400      | Yığın sıfırlandı           |

✓ Tüm MASTER testleri geçti — Errors: 0, Warnings: 4 (zararsız).





Şekil 7.7 — `tb_master` bütünselme testi dalga biçimi (genel görünüm). Saat sinyali, reset sonrası PC'nin sıfırdan başladığını ve her komut için doğru çevrim sayısında ilerlediğini göstermektedir. FSM durum sinyali `S_FETCH` ile başlar; komut tipine göre 3–5 durum arasında gezinip yeniden `S_FETCH`'e döner. `instr_dbg` sinyalinde her komutun doğru opcode'unu taşıdığı görülmektedir. Toplam simülasyon süresi: 6052 ns.

## 7.8 Test sonuçları özeti

Aşağıdaki tablo, projedeki on üç kendi kendini denetleyen testbench'i özetler: sekiz yapı taşı/birim testi (tb\_alu, tb\_alu\_control, tb\_pc, tb\_simple\_register, tb\_instruction\_register, tb\_register\_file, tb\_memory ve tam-CPU bütünleşme testi tb\_cpu) ile beş program/senaryo testi (tb\_itype, tb\_stack, tb\_boundary, tb\_edge, tb\_master). Yukarıdaki §7.1–7.7 başlıkları en kritik testleri dalga biçimleriyle ayrıntılandırır; yapı taşı birim testleri ise ilgili modüllerin yaz/oku ve yükleme davranışını doğrular.

| Testbench               | Amaç                         | Doğrulan                                 | Sonuç |
|-------------------------|------------------------------|------------------------------------------|-------|
| tb_alu                  | ALU birim testi              | 14 assert + zero bayrağı                 | Geçti |
| tb_alu_control          | ALU denetim çözümü           | 15 assert                                | Geçti |
| tb_itype                | I-type aritmetik             | 10 register                              | Geçti |
| tb_stack                | Yığın mekanizması            | 5 register + LIFO + \$sp                 | Geçti |
| tb_boundary             | Sınır durumlar               | 9 register + overflow                    | Geçti |
| tb_edge                 | Uç durumlar                  | 25 register                              | Geçti |
| tb_master               | Kapsamlı bütünleşme          | 18 register (23 komut)                   | Geçti |
| tb_pc                   | Program sayacı birim testi   | PC yükleme, reset ve tutma               | Geçti |
| tb_simple_register      | Genel yazmaç birim testi     | Senkron yükleme ve değer tutma           | Geçti |
| tb_instruction_register | Komut yazmacı birim testi    | IR yükleme ve alan ayrımı                | Geçti |
| tb_register_file        | Register dosyası birim testi | Yaz/oku, \$0 koruması, çok portlu erişim | Geçti |
| tb_memory               | Bellek birim testi           | Word yaz/oku, adres çözümü (byte→word)   | Geçti |
| tb_cpu                  | Tam CPU bütünleşme testi     | 16 register, temel komut akışı           | Geçti |

## 8. Proje Gereksinimlerinin Karşılanması

### *Tüm komutlar test edildi mi?*

Evet. 23 komutun tamamı en az bir testbench'te doğrudan çalıştırılmış ve assert ile doğrulanmıştır. tb\_master tek başına tüm komutları kapsayan bir bütünleşme testidir.

### *Tüm sınır/uç durumlar test edildi mi?*

Evet. tb\_boundary: 32-bit signed overflow, belleğin son geçerli sözcüğü (bayt 1020, word 255), ileri dallanma. tb\_edge: \$0 yazma koruması, negatif immediate, signed karşılaştırma, bgt eşitlik durumu, geri dallanma sayacı, üç katmanlı iç içe push/pop.

### *FSM tüm durumlara girdi mi?*

Evet. tb\_master programı tüm komut tiplerini kapsadığından FSM'deki 23 durumun tamamı en az bir kez ziyaret edilmiştir. tb\_master simülasyonunda state sinyali incelenmiş; S\_FETCH, S\_DECODE, S\_ITYPE\_EX, S\_MEM\_ADDR ve diğer tüm durumlara geçiş gözlemlenmiştir.

### *Yeni komutlar başarıyla entegre edildi mi?*

Evet. Yedi yeni komutun tamamı (mul, swap, loadi, bgt, push, pop, addi3) aşağıdaki katmanlarda tutarlı biçimde gerçekleştirilmiştir:

- `control_unit.vhd` — komuta özel FSM durumları
- `alu.vhd` — OP\_MUL kodu ("1000")
- `datapath.vhd` — ek MUX seçimleri (reg1\_sel, reg2\_sel, alu\_src\_b="100", ...)

- `assembler.py` — komuta özel kodlama dalları
- `tb_stack`, `tb_edge`, `tb_master` — doğrulama

## 9. Sonuç

Bu projede MIPS mimarisini temel alan, FSM denetimli çok çevrimli bir işlemci VHDL ile başarıyla tasarlanmış, yedi yeni komutla genişletilmiş ve kapsamlı bir testbench paketiyle doğrulanmıştır.

İşlemci, 256×32-bit sözcüklük Von Neumann bellek uzayında komut getirme ve veri belleği erişimini doğru biçimde yürütür. 23 durumlu FSM, komut tipine göre 3 ila 5 saat çevrimi arasında çalışarak her komutu farklı sayıda durum üzerinden işler.

Swap komutunun register file'daki oku-önce-yaz özelliği kullanılarak iki ardışık yazma çevrimiyle gerçekleşmesi ve addi3 komutunun ALUOut geri beslemesiyle tek bir veri yolunda üç operandı toplaması, projenin özgün tasarım katkıları arasındadır.

On üç adet kendi kendini denetleyen testbench (8 birim + 5 senaryo) altmıştan fazla register doğrulaması ve sınır/uç durum kontrolünü kapsar. Hiçbir testbench'te hata üretilmemiş; 23 komutun tamamı doğru sonuçlar vermiştir.

## 10. Ekler

### 10.1 Ek A — Tam komut kümesi tablosu

| Komut | Tip | Opcode | Funct | Çevrim | Yeni? |
|-------|-----|--------|-------|--------|-------|
| add   | R   | 0x00   | 0x20  | 4      |       |
| sub   | R   | 0x00   | 0x22  | 4      |       |
| and   | R   | 0x00   | 0x24  | 4      |       |
| or    | R   | 0x00   | 0x25  | 4      |       |
| xor   | R   | 0x00   | 0x26  | 4      |       |
| slt   | R   | 0x00   | 0x2a  | 4      |       |
| mul   | R   | 0x00   | 0x2c  | 4      | ✓     |
| swap  | R   | 0x00   | 0x2d  | 4      | ✓     |
| addi  | I   | 0x08   | —     | 4      |       |
| slti  | I   | 0x0a   | —     | 4      |       |
| andi  | I   | 0x0c   | —     | 4      |       |
| ori   | I   | 0x0d   | —     | 4      |       |
| xori  | I   | 0x0e   | —     | 4      |       |
| loadi | I   | 0x11   | —     | 4      | ✓     |
| bgt   | I   | 0x12   | —     | 3      | ✓     |
| push  | I   | 0x13   | —     | 5      | ✓     |
| pop   | I   | 0x14   | —     | 5      | ✓     |

| Komut | Tip     | Opcod | Funct | Çevrim | Yeni? |
|-------|---------|-------|-------|--------|-------|
| lw    | I       | 0x23  | —     | 5      |       |
| sw    | I       | 0x2b  | —     | 4      |       |
| beq   | I       | 0x04  | —     | 3      |       |
| bne   | I       | 0x05  | —     | 3      |       |
| j     | J       | 0x02  | —     | 3      |       |
| addi3 | Special | 0x10  | —     | 5      | ✓     |

## 10.2 Ek B — Proje dosya yapısı

```

Bilgisayar-Mimarisi-Multi-Cycle-MIPS/
├── src/
│   ├── cpu.vhd           # Üst seviye yapısal modül
│   ├── control_unit.vhd  # FSM tabanlı denetim (23 durum)
│   ├── datapath.vhd      # Veri yolu (MUX + bağlantılar)
│   ├── alu.vhd           # ALU – 7 işlem
│   ├── alu_control.vhd   # ALU opcode çözücü
│   ├── register_file.vhd # 32 × 32-bit register bankası
│   ├── memory.vhd        # 256 × 32-bit Von Neumann bellek
│   ├── pc.vhd            # Program Counter
│   ├── instruction_register.vhd # IR – komut kilidi & alan ayırıcı
│   └── simple_register.vhd # Genel 32-bit register (A,B,ALUOut,MDR)
├── testbench/            (13 self-checking testbench)
├── assembler/assembler.py # İki geçişli Python assembler
└── programs/             (8 ASM + HEX test programı)

```